

BlazingMQ: Business Rules and Semantics

How the broker behaves, written as plain rules

How to read this

These are the rules the BlazingMQ broker follows, written in plain language. They come from the broker source code, not from design documents. Each rule names the file or class it comes from, so you can check it there.

- Rules are numbered by area, for example **D3** is rule 3 under *Delivery modes*.
 - *Primary* is the cluster node that owns a partition and writes to it. *Replica* is a node that keeps a copy. *Proxy* is a broker that forwards traffic to a cluster and stores nothing.
 - *App* (appld) is one named consumer group on a fanout queue.
 - File paths are relative to `src/groups/`.
-

1. Queue lifecycle (Q)

Q1. A queue is named by a URI. The format is `bmq://<domain>[.~tier]/<queue>[?id=<appId>]`. The scheme is always `bmq`. Domain names use `[-a-zA-Z0-9._]` and cannot contain two dots in a row. Queue names use `[-a-zA-Z0-9_~.]` and must be shorter than 64 characters. *Source:* `bmq/bmq/bmq_uri.h` (`bmqt::Uri, k_QUEUE_NAME_MAX_LENGTH = 64`).

Q2. An open must ask to read, write, or both. Opening with neither `READ` nor `WRITE` is invalid. The `ADMIN` flag is only for BlazingMQ admin tasks and is refused for normal clients. `ACK` is an extra flag for writers that want acknowledgements. *Source:* `bmq/bmq/bmq_queueflags.cpp` (`bmqt::QueueFlagsUtil::isValid`).

Q3. A domain that is stopping or being removed refuses new opens. The client gets a `REFUSED` status (“Domain is removing or removed”, or “Node is stopping”). *Source:* `mqb/mqbbbp/mqbbbp_domain.cpp` (`mqbbbp::Domain::openQueue`).

Q4. Domains can cap queues, producers and consumers. `maxQueues`, `maxProducers` and `maxConsumers` reject the open that would go over the limit. `0` (the default) means no limit. *Source:* `mqb/mqbconfm/mqbconf.xsd` (`Domain`); `mqb/mqbbbp/mqbbbp_queueengineutil.h` (`QueueEngineUtil::consumerAndProducerLimitsAreReached`).

Q5. An appId is only allowed on a fanout queue. Opening a priority or broadcast queue with `?id=<appId>` is rejected: “AppId should not be specified when opening a non-fanout queue”. *Source:* `mqb/mqbbbp/mqbbbp_rootqueueengine.cpp` (`RootQueueEngine::configureHandle`).

Q6. A consumer may only read an appId listed in the domain config. A fanout reader with an unknown appId raises the alarm `FANOUT_UNREGISTERED_APPID`. Nothing is delivered to that app, and its confirms are ignored. When its last client leaves, the unknown app is dropped. *Source:* `mqb/mqbbbp/mqbbbp_rootqueueengine.cpp` (`RootQueueEngine, isAuthorized()` checks).

Q7. An open or reopen with nothing to do is refused. If the read, write and admin counts are all zero, the request is not forwarded and fails with `INVALID_ARGUMENT` (“All read,write,admin counts are `<= 0`”). *Source:* `mqb/mqbbbp/mqbbbp_clusterqueuehelper.cpp` (`ClusterQueueHelper, #INVALID_OPENQUEUE_REQ`).

Q8. Every queue lives in exactly one storage partition. The leader picks a partition that has a primary and the fewest queues. If no partition has a primary, it picks the one with the fewest queues. Source: `mqb/mqbc/mqbc_clusterutil.cpp` (`mqbc::ClusterUtil`, queue assignment).

Q9. Only empty, unused queues are garbage collected. A queue is deleted only on its primary, only when it has no handles and no outstanding messages, and only after it has stayed that way for `keepaliveDurationMs` (default 30 minutes). Source: `mqb/mqbblp/mqbblp_clusterqueuehelper.cpp` (`ClusterQueueHelper::gcExpiredQueues`).

Q10. Idle queues can raise an alarm. If `maxIdleTime` is set, an app that has messages waiting but consumes nothing for that many seconds is marked *idle* and an alarm is logged. It goes back to *alive* when it consumes again or has nothing waiting. `0` (default) turns this off. Source: `mqb/mqbblp/mqbblp_queueconsumptionmonitor.h` (`mqbblp::QueueConsumptionMonitor`).

Q11. The routing mode and storage type of a domain cannot change. Reconfiguring a domain from one mode to another (e.g. priority to fanout), or from in-memory to file-backed, is rejected. Source: `mqb/mqbblp/mqbblp_domain.cpp` (`validateConfig: rc_CHANGED_DOMAIN_MODE, rc_CHANGED_STORAGE_TYPE`).

2. Delivery modes (D)

D1. A domain has exactly one mode: priority, fanout, or broadcast. Source: `mqb/mqbconfm/mqbconf.xsd` (`QueueMode`).

D2. Priority: each message goes to one consumer. Only consumers at the highest priority get messages. They share the load round-robin. Lower-priority consumers are standby only. Source: `mqbconf.xsd` (`QueueModePriority`); `mqb/mqbblp/mqbblp_routers.h` (`mqbblp::Routers`).

D3. Fanout: every app gets its own copy. Each `appld` in the domain's `appIDs` list sees every message. Inside one app, the priority rules (D2) pick the consumer. Source: `mqbconf.xsd` (`QueueModeFanout`); `mqb/mqbblp/mqbblp_rootqueueengine.cpp`.

D4. A fanout message stays until every app confirms it. Each app holds its own reference. The message is removed from storage when the last reference is released. Source: `mqb/mqbblp/mqbblp_rootqueueengine.cpp` (`RootQueueEngine::onConfirmMessage, StorageResult::e_ZERO_REFERENCES`).

D5. Broadcast: best effort, at most once. Each message goes to all consumers connected at that moment. It is not tracked, not confirmed and not redelivered. Messages never expire by TTL. Source: `mqbconf.xsd` (`QueueModeBroadcast`); `mqb/mqbblp/mqbblp_queueengineutil.cpp` (`deliverMessageNoTrack`); `mqb/mqbs/mqbs_inmemorystorage.cpp`.

D6. Broadcast domains are always eventual consistency. If a broadcast domain is configured as `strong`, the broker switches it to `eventual` and logs an error. Source: `mqb/mqbblp/mqbblp_domain.cpp` (`normalizeConfig`).

D7. Broadcast producers ask for an ACK every `ackWindowSize` PUTs (default 500). This lets the proxy clear its list of pending broadcast PUTs. Broadcast PUTs are only retransmitted after a `NOT_READY` reply. Source: `mqb/mqbblp/mqbblp_remotequeue.cpp` (`RemoteQueue::postMessage`); `mqbcfg.xsd` (`QueueOperationsConfig`).

D8. A higher priority never spills over. If a message matches a high-priority subscription whose consumers are all full, it waits. It is not sent to a lower-priority consumer. Source: `mqb/mqbblp/mqbblp_routers.cpp` (`Routers::AppContext::selectConsumer`).

3. Subscriptions (S)

S1. A subscription is a boolean expression over message properties. The broker checks it for each message to decide if a consumer should get it. *Source: `bmq/bmqeval/README.dox`; `bmq/bmqeval/bmqeval_simpleevaluator.h` (`bmqeval::SimpleEvaluator`).*

S2. The grammar (from `bmqeval_simpleevaluatorparser.y` and `bmqeval_simpleevaluatorscanner.l`):

Element	Syntax
Property name	starts with a letter, then letters, digits, <code>_</code> or <code>.</code>
Literals	integers (64-bit, may start with <code>-</code>), "strings", <code>true</code> , <code>false</code>
Compare	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>
Logic	<code>&&</code> <code>\ \ </code> <code>!</code> (also <code>~</code>)
Maths	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> and unary <code>-</code>
Test	<code>exists(name)</code>
Group	<code>(...)</code>

Precedence, lowest to highest: `||`, `&&`, equality, comparison, `+` `-`, `*` `/` `%`, then `!` and unary `-`.

S3. Expressions have size limits. At most 128 characters, 10 operators and 10 property references. An integer that does not fit in 64 bits is an “integer overflow” error. *Source: `bmqeval_simpleevaluator.h` (`k_MAX_EXPRESSION_LENGTH`, `k_MAX_OPERATORS`, `k_MAX_PROPERTIES`; errors `e_T00_LONG`, `e_T00_COMPLEX`, `e_SYNTAX`); `bmqeval_simpleevaluatorparser.y`.*

S4. An expression must use at least one property. `true` on its own is rejected (`e_NO_PROPERTIES`). *Source: `bmqeval_simpleevaluator.h` (`ErrorType::e_NO_PROPERTIES`).*

S5. Any evaluation error means “no match”. A missing property, a wrong type, or a result that is not boolean makes the expression false. The message is not delivered to that subscription. *Source: `mqb/mqbbbp/mqbbbp_routers.cpp` (`Routers::Expression::evaluate`).*

S6. An expression that is missing or did not compile matches everything. The router only filters with a valid compiled expression; otherwise it lets the message through. This is why config validation (S7) matters. *Source: `mqb/mqbbbp/mqbbbp_routers.cpp` (`Routers::Expression::evaluate truth table`).*

S7. Bad domain subscriptions block the config. Each app subscription in a domain config is compiled on load; any invalid one rejects the whole domain config. *Source: `mqb/mqbbbp/mqbbbp_domain.cpp` (`validateConfig`, `rc_INVALID_SUBSCRIPTION`).*

S8. If no app’s domain subscription matches, the message is not stored. The producer still gets a success ACK. *Source: `mqb/mqbbbp/mqbbbp_localqueue.cpp` (`LocalQueue::postMessage`); `RootQueueEngine::messageReferenceCount`.*

S9. A message no one can take is put aside, not dropped. If no subscription matches or the matching consumers are full, the message is set aside and the next messages are tried. *Source: `mqb/mqbbbp/mqbbbp_queueengineutil.cpp` (`QueueEngineUtil_AppState::processDeliveryList`, `putAside`).*

S10. Message property limits. Up to 255 properties per message, names up to 4,095 bytes, and the whole property area just under 64 MB. Types: `bool`, `char`, `short`, `int32`, `int64`, `string`, `binary`. *Source: `bmq/bmq/bmq_messageproperties.h` (`k_MAX_NUM_PROPERTIES`, `k_MAX_PROPERTY_NAME_LENGTH`); `bmq/bmq/bmq_propertytype.h` (`bmq::PropertyType`).*

4. ACK, CONFIRM and redelivery (A)

A1. ACK goes to the producer, CONFIRM comes from the consumer. ACK says the broker accepted (or refused) a PUT. CONFIRM says the consumer is done with a message.

A2. Eventual consistency: ACK once the primary has written the message. *Source:* `mqb/mqbs/mqbs_filestore.cpp` (receipt is set at once when `d_replicationFactor == 1`).

A3. Strong consistency: ACK only after a quorum of nodes has the message. The primary waits for replication receipts from $(\text{nodes} / 2) + 1$ nodes before it ACKs or delivers. *Source:* `mqbconf.xsd` (Consistency); `mqb/mqbc/mqbc_storagemanager.cpp` (`d_replicationFactor`); `mqb/mqbs/mqbs_filestore.cpp` (`processReceiptEvent`); `mqb/mqbbbp/mqbbbp_localqueue.cpp` (`onReceipt`).

A4. A failed PUT always gets a NACK, even if no ACK was requested. Reasons map to `bmqt::AckResult`:

Storage result	ACK result
queue message limit reached	LIMIT_MESSAGES
queue byte limit reached	LIMIT_BYTES
disk write failed	STORAGE_FAILURE
duplicate GUID	SUCCESS (silently de-duplicated)
any other error	UNKNOWN

Source: `mqb/mqbi/mqbi_storage.cpp` (`StorageResult::toAckResult`); `mqbbbp_localqueue.cpp`.

A5. PUT from a client that did not open for WRITE is refused (REFUSED, logged as `#CLIENT_IMPROPER_BEHAVIOR`). *Source:* `mqb/mqbbbp/mqbbbp_localqueue.cpp` (`LocalQueue::postMessage`).

A6. Duplicate PUTs are dropped for deduplicationTimeMs (default 5 minutes). A PUT whose GUID was seen in that window is not stored again. *Source:* `mqbconf.xsd` (`deduplicationTimeMs`); `mqb/mqbs/mqbs_filebackedstorage.cpp`, `mqbs_inmemorystorage.cpp` (`e_DUPLICATE`).

A7. Unconfirmed messages are redelivered when a consumer goes away. They go on the app's redelivery list, which is drained before any new messages. A new primary redelivers everything that was not confirmed. *Source:* `mqb/mqbbbp/mqbbbp_queueengineutil.h` (`QueueEngineUtil_AppState`, redelivery list); `mqbbbp_rootqueueengine.cpp` ("Primary redelivers everything").

A8. A consumer crash counts as a delivery attempt. For each unconfirmed message of a crashed client, the broker *rejects* it, which lowers its remaining-attempts counter by one. *Source:* `mqb/mqbbbp/mqbbbp_queuehandle.cpp` (`QueueHandle::clearClient`); `RootQueueEngine::onRejectMessage`.

A9. Poison messages are purged after maxDeliveryAttempts. When the counter hits zero, the message is auto-confirmed for that app, dumped to a temp file and an alarm is raised. 0 (default) means unlimited. *Source:* `mqbconf.xsd` (`maxDeliveryAttempts`); `RootQueueEngine::onRejectMessage`; `QueueEngineUtil::logRejectMessage`.

A10. Changing maxDeliveryAttempts updates stored messages only between limited and unlimited. Limited-to-limited changes apply to new messages only. *Source:* `RootQueueEngine::onRejectMessage` (truth table in code comment).

A11. Possibly-poisonous messages are redelivered with a delay. Once a message has been rejected, redelivery waits `highInterval` (default 3 s) when few attempts remain (counter at or below `lowThreshold`, default 2), and `lowInterval` (default 1 s) when the counter is above `lowThreshold` and at or below `highThreshold` (default 4). Above `highThreshold` there is no delay. *Source:* `mqb/mqbbbp/mqbbbp_queueengineutil.cpp` (`QueueEngineUtil::loadMessageDelay`); `mqbcfg.xsd` (`MessageThrottleConfig`).

A12. Confirms that cannot be applied are ignored. A confirm for an unregistered app is dropped. A confirm for a message the storage cannot find is logged and has no effect. *Source:* `RootQueueEngine::onConfirmMessage`.

A13. Broadcast queues have no CONFIRM or reject. (See D5.) *Source:* `RootQueueEngine::onConfirmMessage`, `onRejectMessage` (asserts not broadcast).

5. Capacity and flow control (C)

C1. Each consumer has an in-flight limit. By default a subscription may hold 1,000 unconfirmed messages or 32 MB (33,554,432 bytes), whichever comes first. Then the broker stops sending to it. *Source:* `bmq/bmq/bmq_queueoptions.cpp`, `bmqt_subscription.cpp` (`k_DEFAULT_MAX_UNCONFIRMED*`).

C2. Delivery resumes at 80% of that limit. After hitting the limit, the consumer becomes usable again when confirms bring it back down to the low watermark. *Source:* `mqb/mqbbbp/mqbbbp_queuehandle.cpp` (`k_WATERMARK_RATIO = 0.8`, `d_unconfirmedMonitor`).

C3. Consumer priority range. From `INT_MIN/2` to `INT_MAX/2`; default 0. *Source:* `bmqt_queueoptions.cpp` (`k_CONSUMER_PRIORITY_MIN/MAX`, `k_DEFAULT_CONSUMER_PRIORITY`).

C4. Storage has two levels of limits: per queue and per domain. Each has a message count and a byte count. The queue meter is a child of the domain meter, so a PUT must fit in both. *Source:* `mqbconf.xsd` (`StorageDefinition: domainLimits`, `queueLimits`); `mqb/mqbu/mqbu_capacitymeter.h`.

C5. Over the limit, the PUT is NACKed with `LIMIT_MESSAGES` or `LIMIT_BYTES` (A4). *Source:* `mqbs_filebackedstorage.cpp`, `mqbs_inmemorystorage.cpp` (`commitUnreserved`).

C6. A high-watermark alarm fires at 80% of a limit by default (`messagesWatermarkRatio`, `bytesWatermarkRatio`). Each ratio must be between 0 and 1. *Source:* `mqbconf.xsd` (`Limits`); `mqb/mqbbbp/mqbbbp_domain.cpp` (`rc_WRONG_WATERMARK_RATIO`).

C7. Message TTL is a minimum, not an exact time. A message is never removed before `messageTtl` seconds, but may stay a little longer. *Source:* `mqbconf.xsd` (`messageTtl`); `mqbs_inmemorystorage.cpp`, `mqbs_filebackedstorage.cpp` (`gcExpiredMessages`).

C8. Payload limit is 64 MB per PUT or PUSH. An empty payload is rejected (`PAYLOAD_EMPTY`). *Source:* `bmq/bmq/bmq_protocol.h` (`PutHeader::k_MAX_PAYLOAD_SIZE_SOFT`); `bmq/bmq/bmq_resultcode.h` (`EventBuilderResult`).

C9. Proxies do not hand messages back. A proxy keeps what upstream sent it and spreads it over its own readers, even if the reader that asked for them has left. *Source:* `mqb/mqbbbp/mqbbbp_relayqueueengine.h` (`mqbbbp::RelayQueueEngine`, "Known issues").

6. Persistence and recovery (P)

P1. Each node splits storage into numPartitions partitions. Each partition has a primary that is the only writer. *Source:* `mqbcfg.xsd` (`PartitionConfig`); `mqb/mqbc/mqbc_storagemanager.h`.

P2. A partition is a set of files: journal, data, and (legacy) QLIST. Each file has a max size from config. The cluster state ledger (CSL) file defaults to 64 MB. *Source:* `mqbcfg.xsd` (`maxDataFileSize`, `maxJournalFileSize`, `maxQlistFileSize`, `maxCSLFileSize`); `mqb/mqbs/mqbs_filestore.h`.

P3. When a file fills up, the partition rolls over. Only messages still outstanding are copied to a new file set. The old set is archived; at most `maxArchivedFileSets` are kept. *Source:*

`mqb/mqbs/mqbs_filestore.h` (`FileStore::rolloverImpl`, `rolloverIfNeeded`).

P4. Sync points mark safe recovery positions. The primary writes them regularly. Recovery and rollover line up on them. Source: `mqb/mqbs/mqbs_filestore.h` (`issueSyncPoint`, `d_firstSyncPointAfterRolloverSeqNum`).

P5. Every write has a Partition Sequence Number (PSN) = primary lease id + sequence number. A higher lease means a newer primary. Nodes compare PSNs to find who is most up to date. Source: `mqb/mqbc/mqbc_partitionfsm.h` (`PartitionFSMEventData`); `bmqp_ctrlmsg::PartitionSequenceNumber`.

P6. On startup a node recovers each partition from a peer if a primary exists, or from its own files if the whole cluster is starting together. It waits `startupWaitDurationMs` (default 60 s) for a sync point first. Recovery must finish within `startupRecoveryMaxDurationMs` (default 20 min), with up to `maxAttemptsStorageSync` (3) tries. Source: `mqbcfg.xsd` (`StorageSyncConfig`); `mqb/mqbc/mqbc_recoverymanager.h`.

P7. In-memory domains are not persisted. Only `fileBacked` storage survives a restart. Source: `mqbconf.xsd` (`Storage: inMemory`, `fileBacked`).

P8. Files are flushed to disk on shutdown unless `flushAtShutdown` is false. Source: `mqbcfg.xsd` (`PartitionConfig.flushAtShutdown`, default `true`).

7. Replication and leader election (R)

R1. One leader per cluster, chosen by a Raft-style vote. A candidate needs `quorum` votes. If `quorum` is 0 (default), it is half the nodes plus one. This guarantees at most one leader, even during a network split. Source: `mqb/mqbnnet/mqbnnet_elector.h` (`mqbnnet::Elector`); `mqbcfg.xsd` (`ElectorConfig.quorum`).

R2. Node election states: DORMANT, FOLLOWER, CANDIDATE, LEADER. A follower starts an election when the leader misses `heartbeatMissCount` (10) heartbeats. Each node waits a random time first, so they do not all run at once. Source: `mqbnnet_elector.h` (`ElectorState`, `ElectorTimerEventType`); `mqbcfg.xsd` (`ElectorConfig`).

R3. Before an election, a node scouts. It asks peers if they would vote, and only proposes if enough say yes. Source: `mqbnnet_elector.h` (`e_SCOUTING_REQUEST`, `e_SCOUTING_RESPONSE`).

R4. Only the leader writes cluster state. The leader applies changes to its ledger (CSL), broadcasts them, and followers apply them. A change is committed when the configured consistency is reached: *eventual* (no follower ACKs) or *strong* (majority ACKs). Source: `mqb/mqbc/mqbc_clusterstateledger.h` (`ClusterStateLedger`, `ClusterStateLedgerConsistency`).

R5. A new leader heals the cluster first. It collects followers' ledger positions (LSNs), fetches the newest ledger if someone is ahead, then commits a leader advisory. States: `LDR_HEALING_STG1/2/3` then `LDR_HEALED`; followers go `FOL_WAITING`, `FOL_HEALING`, `FOL_HEALED`. Source: `mqb/mqbc/mqbc_clusterstatetable.h` (`ClusterStateTableState`); `mqb/mqbc/mqbc_clusterfsm.h`.

R6. The leader picks partition primaries. The supported algorithm is `leaderIsMasterAll`: the leader is primary for all partitions. `leastAssigned` is not supported yet; it and unknown values fall back to `leaderIsMasterAll` with an alarm. Source: `mqb/mqbc/mqbc_clusterutil.cpp` (`getNextPrimarys`, `assignPartitions`).

R7. A new primary heals its partition before serving. It asks replicas for their PSN, syncs with the most up-to-date node if it is behind, then pushes data to replicas. Only then is it `PRIMARY_HEALED`, which means a quorum has the same data. Source: `mqb/mqbc/mqbc_partitionstatetable.h` (`PartitionStateTableState`); `mqb/mqbc/mqbc_partitionfsm.h`.

R8. A waiting node rejects a second heal request. `FOL_WAITING` and `REPLICA_WAITING` must reject new requests so the same node is not healed twice in a row. *Source:* `mqbc_clusterstatetable.h`, `mqbc_partitionstatetable.h` (state comments).

R9. Stuck state machines kill the broker. If the cluster or partition FSM does not settle within its watchdog timeout (default 5 min) after the allowed retries (1), the broker stops. *Source:* `mqbcfg.xsd` (`ClusterAttributes.*FsmWatchdog*`); `mqb/mqbc/mqbc_watchdogcontext.h`.

R10. FSM mode needs CSL mode. `isFSMWorkflow` must be false when `isCSLModeEnabled` is false. *Source:* `mqbcfg.xsd` (`ClusterAttributes`).

8. Configuration constraints and defaults (K)

Domain config (`mqb/mqbconfm/mqbconf.xsd`, `Domain`, validated by `mqbblp::Domain`):

Setting	Default	Rule
<code>maxConsumers</code> / <code>maxProducers</code> / <code>maxQueues</code>	0	0 = unlimited
<code>maxIdleTime</code>	0 s	0 = no idle alarm
<code>maxDeliveryAttempts</code>	0	0 = unlimited
<code>deduplicationTimeMs</code>	300,000 (5 min)	
<code>messagesWatermarkRatio</code> / <code>bytesWatermarkRatio</code>	0.8	must be in [0, 1]
<code>msgGroupIdConfig.maxGroups</code>	2,147,483,647	least recently used group evicted
<code>msgGroupIdConfig.ttlSeconds</code>	0	0 = unlimited
<code>fanout.publishAppIdMetrics</code>	true	
<code>mode</code> , storage type	(required)	cannot change after first config
<code>consistency</code>		broadcast is forced to eventual

Client queue options (`bmq/bmq/bmq_queueoptions.cpp`):

Setting	Default
<code>maxUnconfirmedMessages</code>	1,000
<code>maxUnconfirmedBytes</code>	33,554,432 (32 MB)
<code>consumerPriority</code>	0 (range <code>INT_MIN/2</code> .. <code>INT_MAX/2</code>)
<code>suspendsOnBadHostHealth</code>	false

Cluster queue operations (`mqb/mqbcfg/mqbcfg.xsd`, `QueueOperationsConfig`):

Setting	Default	Rule
openTimeoutMs / configureTimeoutMs / closeTimeoutMs	300,000 (5 min)	configure must be $\leq$ close
reopenTimeoutMs	43,200,000 (12 h)	long on purpose, survives outages
reopenRetryIntervalMs / reopenMaxAttempts	5,000 / 10	
assignmentTimeoutMs	15,000	
keepaliveDurationMs	1,800,000 (30 min)	see Q9
consumptionMonitorPeriodMs	30,000	
stopTimeoutMs / shutdownTimeoutMs	10,000 / 20,000	shutdown must be $>$ stop
ackWindowSize	500	see D7

Elector (mqbcfg.xsd, ElectorConfig):

Setting	Default	Rule
initialWaitTimeoutMs	8,000	must be $>$ maxRandomWaitTimeoutMs
maxRandomWaitTimeoutMs	3,000	
scoutingResultTimeoutMs / electionResultTimeoutMs	4,000 / 4,000	
heartbeatBroadcastPeriodMs / heartbeatCheckPeriodMs	2,000 / 1,000	
heartbeatMissCount	10	
quorum	0	$0 = \text{nodes}/2 + 1$
leaderSyncDelayMs	80,000	clusters with more than 1 node

Storage and sync (mqbcfg.xsd, PartitionConfig, StorageSyncConfig):

Setting	Default
maxCSLFileSize	67,108,864 (64 MB)
preallocate / predefaultPages / flushAtShutdown	false / false / true
startupRecoveryMaxDurationMs	1,200,000 (20 min)
maxAttemptsStorageSync	3
storageSyncReqTimeoutMs	300,000 (5 min)
masterSyncMaxDurationMs	600,000 (10 min)
partitionSync{State,Data}ReqTimeoutMs	120,000 (2 min)
startupWaitDurationMs	60,000
fileChunkSize / partitionSyncEventSize	4 MB / 4 MB

Poison-message throttling (mqbcfg.xsd, MessageThrottleConfig): lowThreshold 2, highThreshold 4, lowInterval 1,000 ms, highInterval 3,000 ms. lowThreshold must be below highThreshold, and lowInterval at most highInterval.

Network (mqbcfg.xsd, TcpInterfaceConfig, Heartbeat): maxConnections 10,000, heartbeatIntervalMs 3,000. Missed-heartbeat limits per connection type default to 0, which turns smart-heartbeat off. Client default broker port is 30114 (bmq/bmqt/bmqt_sessionoptions.h).

Appendix: result codes a client sees

From bmq/bmqt/bmqt_resultcode.h. Negative = error, positive = warning, 0 = success.

Code family	Values
Generic	SUCCESS 0, UNKNOWN -1, TIMEOUT -2, NOT_CONNECTED -3, CANCELED -4, NOT_SUPPORTED -5, REFUSED -6, INVALID_ARGUMENT -7, NOT_READY -8
Open queue	ALREADY_OPENED 100, ALREADY_IN_PROGRESS 101, INVALID_URI -100, INVALID_FLAGS -101, CORRELATIONID_NOT_UNIQUE -102
Close queue	ALREADY_CLOSED 100, ALREADY_IN_PROGRESS 101, UNKNOWN_QUEUE -100, INVALID_QUEUE -101
Post (event builder)	QUEUE_INVALID -100, QUEUE_READONLY -101, MISSING_CORRELATION_ID -102, EVENT_TOO_BIG -103, PAYLOAD_TOO_BIG -104, PAYLOAD_EMPTY -105, OPTION_TOO_BIG -106, QUEUE_SUSPENDED -108
ACK	LIMIT_MESSAGES -100, LIMIT_BYTES -101, LIMIT_QUEUE_MESSAGES -102, LIMIT_QUEUE_BYTES -103, STORAGE_FAILURE -104 (plus generic)